



Matteo Imbrosciano

1000014829

[Git Hub](#)

Shop-Cloud

Prof: Giuseppe Pappalardo

Prof: Andrea Fornaia

1 – Obiettivi

L'obiettivo del progetto è di sviluppare una piattaforma di shop su cloud che consenta agli utenti di acquistare prodotti come T-Shirt, Scarpe, Pantaloni.

- **Visualizzazione degli Prodotti:** Gli utenti devono essere in grado di visualizzare una lista completa dei prodotti(T-Shirt, Scarpe, Pantaloni)
- **Aggiunta al Carrello:** Gli utenti devono poter aggiungere articoli al carrello per procedere con l'acquisto. Questa funzionalità facilita la selezione e la gestione dei prodotti da acquistare.
- **Ricerca dei Prodotti:** Gli utenti devono poter cercare i prodotti(T-Shirt, Scarpe, Pantaloni) utilizzando parole chiave per trovare rapidamente gli articoli di loro interesse. Questa funzionalità migliora l'usabilità e facilita la navigazione nel catalogo prodotti.
- **Memorizzazione dello Stato del Carrello:** Il sistema deve memorizzare lo stato del carrello dell'utente per consentire l'accesso e la gestione dei dati degli acquisti in sessioni future.

I microservizi che compongono l'applicazione saranno containerizzati e deployati su un cloud provider. La containerizzazione semplifica il processo di deploy, permettendo di creare ambienti isolati e replicabili, riducendo così il rischio di conflitti tra dipendenze e facilitando la gestione delle versioni e degli aggiornamenti.

Al fine di gestire in modo efficace il ciclo di vita del software, sarà implementata una pipeline CI/CD utilizzando le GitHub Actions. Le pipeline automatizzeranno il processo integrazione e deployment dei microservizi.

2 – Architettura e Tecnologie

Il progetto si propone di realizzare una piattaforma di shop per la vendita di prodotti come T-Shirt, Scarpe e Pantaloni, strutturata su tre microservizi sviluppati in Java utilizzando il framework Spring Boot e gestiti tramite Maven. I microservizi saranno containerizzati utilizzando Docker e orchestrati con Kubernetes, per essere poi distribuiti e gestiti su un'infrastruttura cloud.

Microservizi:

- **Client:** permette all'utente di inserire il nome con cui sarà salvata la sessione. Contiene il Circuit Breaker tramite il quale effettuiamo le chiamate REST API. Il Circuit Breaker ha un riferimento alla classe `HttpRequest` per mandare le richieste al server. Il client può visualizzare la lista dei prodotti (T-Shirt, Scarpe, Pantaloni) disponibili, può cercare un prodotto scrivendo una parola contenuta nel nome di esso, può acquistare un prodotto e visualizzare la lista dei suoi acquisti.
- **Server:** riceve le richieste dal Client. Il Server contiene i dati dei prodotti in vendita e permette la ricerca dei prodotti. In caso che il client chiede di visualizzare il suo carrello o chiede di acquistare un prodotto il Server inoltra la richiesta al microservizio che si occupa di gestire la sessione.
- **Sessione:** si occupa di gestire la sessione dei client. Contiene una mappatura in cui ogni nome del client è associata la lista degli acquisti effettuati. Lo stato viene serializzato e salvato nel file system del microservizio così da salvare i dati dell'utente anche quando esso non è collegato.

Funzionamento del sistema

1. Gli utenti interagiscono con il sistema tramite il microservizio Client, che funge da punto di ingresso per tutte le richieste.
2. Il Client invia richieste al Server, che si occupa della gestione dei dati dei prodotti, elaborando richieste di ricerca o gestione degli acquisti.
3. Per operazioni legate alla sessione utente, il Server si interfaccia con il microservizio Sessione, che archivia e recupera le informazioni relative agli acquisti di ciascun utente.

Docker

La containerizzazione dei microservizi è stata ottenuta attraverso la creazione di immagini Docker specifiche per ciascuno dei tre servizi coinvolti. Per ciascun servizio, è stato realizzato un file di configurazione denominato Dockerfile, che funge da guida per creare un'immagine Docker personalizzata.

I Dockerfile creano le immagini Docker per le applicazioni Java. Si parte da un ambiente Maven già configurato, copia i file del progetto all'interno dell'immagine, costruisce l'applicazione e alla fine esegue il packaging dell'applicazione Java.

Ogni Dockerfile non si limita solo a costruire l'applicazione, ma assicura che tutte le dipendenze siano risolte e che l'ambiente di runtime sia configurato correttamente. Questo include l'installazione dei pacchetti necessari, la configurazione di librerie specifiche e l'impostazione dei comandi per avviare il servizio all'interno del container.

Una volta create le immagini Docker, queste sono state testate localmente. I test hanno verificato che ogni microservizio funzionasse correttamente quando eseguito all'interno di un container Docker.

Questo passaggio ha incluso controlli per garantire:

- **Compatibilità del microservizio con il container:** Ogni applicazione doveva funzionare senza problemi nell'ambiente isolato fornito da Docker;
- **Correttezza della gestione delle dipendenze:** Tutti i pacchetti richiesti dovevano essere disponibili e correttamente configurati;
- **Configurazione dell'ambiente di runtime:** Variabili di ambiente, porte di rete e altri parametri dovevano essere impostati correttamente.

Ogni microservizio è stato racchiuso in un container Docker, che fornisce un ambiente isolato e coerente per l'esecuzione dell'applicazione. Questo ha incluso la verifica della compatibilità, la gestione delle dipendenze e la configurazione dell'ambiente di runtime.

Dopo aver verificato il funzionamento locale dei microservizi nei container, è stato necessario rendere le immagini Docker disponibili per il deployment su una macchina virtuale EC2 (fornita da Amazon Web Services). Questo è stato realizzato effettuando il **push delle immagini Docker su Docker Hub**, una piattaforma pubblica per il versionamento e la distribuzione di immagini Docker. Docker Hub ha consentito di centralizzare le immagini, rendendole facilmente accessibili dalla macchina EC2, che ha potuto scaricarle ed eseguirle direttamente.

Kubernetes

Ho orchestrato i microservizi del progetto Shop-Cloud utilizzando Kubernetes, un potente sistema open-source progettato per automatizzare la gestione, il deployment e la scalabilità dei container. Questa scelta ha garantito che i microservizi potessero essere gestiti in modo efficiente e scalabile all'interno di un'infrastruttura cloud, fornendo un ambiente robusto e flessibile per il progetto.

L'orchestrazione con Kubernetes ha comportato la suddivisione dei microservizi in deployment separati. Ogni deployment rappresenta una configurazione per la creazione e la gestione di istanze specifiche di un determinato microservizio.

In particolare, ho gestito tre microservizi distinti:

- **Client:** Responsabile dell'interfaccia utente e delle richieste degli utenti finali;
- **Server:** Il cuore della logica applicativa, che elabora le richieste provenienti dal client e interagisce con i dati;
- **Session:** Un servizio dedicato alla gestione delle sessioni degli utenti, assicurando che i dati di sessione siano sincronizzati e accessibili.

Ogni deployment è stato configurato per consentire la **replicazione, aggiornamento e monitoraggio indipendente** di ciascun microservizio.

Ciò significa che Kubernetes può gestire automaticamente più repliche dei container, garantendo l'alta disponibilità dei servizi. Inoltre, Kubernetes consente aggiornamenti progressivi, riducendo al minimo i tempi di inattività e i rischi associati.

Per indirizzare il traffico verso i microservizi, ho configurato i Service in Kubernetes. Un Service è un'astrazione che collega i pod (le unità esecutive di Kubernetes che contengono i container) di un determinato microservizio a un endpoint stabile.

Questi endpoint:

- Facilitano la comunicazione interna tra i componenti del sistema.
- Espongono i microservizi al traffico esterno (quando necessario) attraverso indirizzi IP o nomi DNS stabili, indipendentemente dalle variazioni nei pod sottostanti.

Sul fronte del networking, Kubernetes ha gestito in modo efficace la comunicazione tra i microservizi all'interno del cluster. Questo è stato possibile grazie al sistema di networking interno di Kubernetes, che utilizza nomi di servizio DNS per la risoluzione degli indirizzi.

Questo approccio ha eliminato la necessità di configurare manualmente indirizzi IP statici per i singoli microservizi, semplificando enormemente la gestione della rete.

Inoltre:

- Il networking interno ha garantito una comunicazione affidabile e sicura tra i vari microservizi;
- È stato migliorato il livello di resilienza, poiché eventuali cambiamenti nella topologia del cluster non hanno avuto alcun impatto sulla risoluzione degli indirizzi.

In sintesi, l'adozione di Kubernetes ha portato un alto grado di automazione e scalabilità al sistema, consentendo di orchestrare i microservizi in modo efficiente e garantendo che il progetto potesse soddisfare le esigenze di alta disponibilità, gestione dinamica e semplicità operativa in un ambiente cloud.

Cloud AWS

Ho effettuato il deployment dell'applicazione di Shop-Cloud su Amazon Web Services (AWS), utilizzando macchine virtuali EC2 di tipo t3.medium. Questo tipo di istanza è stato selezionato attentamente per il suo equilibrio tra costi e prestazioni, garantendo una configurazione in grado di supportare efficacemente i carichi di lavoro richiesti dal sistema.

Le istanze t3.medium offrono una combinazione ideale di CPU, memoria e capacità di rete, caratteristiche essenziali per applicazioni basate su web e microservizi. In particolare, queste istanze sono dotate di:

- **2 vCPU (virtual CPU):** sufficiente per gestire le richieste simultanee provenienti dagli utenti e per elaborare le operazioni necessarie.
- **4 GB di memoria RAM:** adeguati a gestire l'esecuzione dei microservizi containerizzati e a garantire prestazioni stabili.
- **Modello di CPU basato su crediti:** Questo sistema permette di accumulare crediti di CPU durante periodi di bassa attività e di spenderli durante i picchi di utilizzo, garantendo così elevate prestazioni quando necessarie. Allo stesso tempo, questa configurazione aiuta a mantenere i costi operativi contenuti, soprattutto durante i periodi di minor carico.

Per garantire la sicurezza e la comunicazione efficiente tra i vari componenti del sistema, ho creato una Virtual Private Cloud (VPC) dedicata per il progetto. La configurazione della VPC include:

- **Un intervallo di indirizzi IP privati:** Ho scelto un intervallo di indirizzi CIDR (Classless Inter-Domain Routing) che fosse sufficientemente ampio per supportare tutti i microservizi e i loro componenti all'interno della rete privata. Questo ha consentito di assegnare indirizzi IP univoci a ciascun componente senza conflitti.
- **Sottoreti (subnet):** La rete è stata suddivisa in sottoreti private e pubbliche, con i servizi più sensibili ospitati in subnet private per motivi di sicurezza.

Per gestire il traffico di rete, ho configurato specifiche **Route Tables**:

- **Traffico interno alla VPC:** Le tabelle di routing assicurano che i vari microservizi possano comunicare tra loro all'interno della rete privata in modo rapido e sicuro.
- **Accesso a Internet:** Ho configurato un Internet Gateway (IGW) associato alla VPC. Questo gateway consente alle risorse della subnet privata di accedere a Internet quando necessario, ad esempio per:
 - Scaricare aggiornamenti software.
 - Effettuare chiamate a servizi esterni o API.

Benefici di utilizzare AWS:

- **Scalabilità e prestazioni:** Le istanze EC2 di tipo t3.medium forniscono un'infrastruttura affidabile per gestire i carichi di lavoro, adattandosi a variazioni nelle richieste grazie al modello di crediti di CPU;
- **Isolamento e sicurezza:** La configurazione della VPC con indirizzi IP privati e subnet dedicate garantisce che i microservizi siano isolati dalle minacce esterne, riducendo il rischio di accessi non autorizzati;
- **Ottimizzazione dei costi:** L'uso di un modello a crediti e l'accesso controllato a Internet consentono di ottimizzare i costi operativi, mantenendo comunque elevate le prestazioni nei momenti critici.

In sintesi, l'architettura del deployment su AWS è stata progettata per offrire sicurezza, affidabilità e scalabilità, assicurando che l'applicazione possa gestire carichi di lavoro variabili in modo efficiente, con costi contenuti.

CI/CD

L'applicazione è stata distribuita utilizzando un processo completamente automatizzato tramite GitHub Actions, una piattaforma che consente di configurare e gestire pipeline per l'integrazione e il deployment continuo (CI/CD). Questo approccio ha semplificato e reso più efficiente l'intero ciclo di vita dello sviluppo, dalla scrittura del codice alla distribuzione in produzione.

Workflow CI/CD automatizzato

Ogni volta che viene effettuato un push o una pull request al repository GitHub, viene attivato un workflow configurato in un file YAML all'interno del repository. Questo workflow include una serie di passaggi automatizzati, tra cui:

1. Build delle immagini Docker:

- Il workflow avvia la creazione delle immagini Docker per l'applicazione utilizzando i file Dockerfile presenti nel repository;
- Durante questa fase, vengono installate le dipendenze, configurato l'ambiente e impacchettata l'applicazione in container eseguibili.

2. Push delle immagini su Docker Hub:

Dopo aver costruito le immagini Docker, queste vengono automaticamente caricate sul relativo repository su Docker Hub. Questo passaggio assicura che le immagini siano centralizzate e facilmente accessibili da qualsiasi ambiente, inclusa l'infrastruttura cloud.

3. Deployment automatico su EC2:

- Una volta che le immagini Docker sono disponibili su Docker Hub, il workflow si occupa di distribuire l'applicazione sulle istanze EC2 configurate in precedenza;
- Questo include il pull delle immagini dal Docker Hub, l'esecuzione dei container e la gestione delle configurazioni necessarie per l'avvio dell'applicazione.

Vantaggi del processo automatizzato

L'implementazione di un sistema CI/CD tramite GitHub Actions ha portato numerosi vantaggi operativi:

- **Distribuzione rapida e continua:** Ogni modifica al codice viene automaticamente integrata e distribuita. Questo garantisce che le nuove funzionalità, correzioni di bug e aggiornamenti siano disponibili in produzione nel minor tempo possibile.
- **Riduzione degli errori manuali:** Il processo automatizzato elimina la necessità di interventi manuali, riducendo così il rischio di errori umani durante la build o il deployment.
- **Maggiore coerenza:** Il workflow garantisce che tutte le modifiche siano sottoposte allo stesso processo di build e deployment, assicurando un ambiente uniforme e prevedibile.
- **Ottimizzazione del time-to-market:** Grazie all'automazione, i tempi necessari per portare nuove funzionalità in produzione sono significativamente ridotti, migliorando la competitività del prodotto.

Come funziona la pipeline CI/CD

- **Trigger degli eventi:** La pipeline si avvia automaticamente ogni volta che viene effettuato un push, una pull request o qualsiasi altro evento definito nelle configurazioni del workflow GitHub Actions.
- **Esecuzione dei job:** I job definiti nel workflow includono step specifici come la build delle immagini Docker, i test automatizzati e il deployment sulle istanze EC2.
- **Notifiche e monitoraggio:** Il sistema invia notifiche sullo stato del workflow (ad esempio, successo o fallimento), permettendo al team di monitorare l'intero processo e intervenire rapidamente in caso di problemi.

Al fine di permettere il deploy automatico ho aggiunto le seguenti directory:

Shop-cloud/

```
|— .github/
|   |— workflows/
|       |— docker-publish.yaml
|       |— ec2-deploy.yaml
|— scripts/
|   |— deploy1.sh
|   |— deploy2.sh
```

L'utilizzo di **GitHub Actions** per l'automazione del processo di CI/CD ha reso lo sviluppo e la distribuzione dell'applicazione più veloci, affidabili ed efficienti. Questo sistema permette di reagire prontamente alle modifiche del codice, assicurando che l'applicazione sia sempre aggiornata e operativa, riducendo i costi operativi e accelerando il rilascio di nuove funzionalità.

Il file **docker-publish.yaml** gestisce il processo di autenticazione su Docker Hub utilizzando le variabili segrete `DOCKER_USERNAME` e `DOCKER_PASSWORD`. Successivamente, costruisce le immagini Docker per i tre microservizi e le carica nel repository Docker Hub.

Il file **ec2-deploy.yaml** contiene il trigger che avvia l'automazione del processo di Continuous Deployment (CD). Questo trigger si attiva ogni volta che viene effettuato un push sul branch "main". Ogni modifica al codice che viene inviata al branch **main** del repository avvia automaticamente il workflow di deployment, garantendo che le nuove versioni del codice vengano distribuite sulla macchina EC2 designata. Dopo aver configurato l'autenticazione SSH, gli script vengono copiati dalla directory del repository locale alla directory `/home/ec2-user/` sulla macchina EC2 utilizzando `scp`. Successivamente, questi script vengono eseguiti in sequenza sulla macchina remota tramite SSH, garantendo che le operazioni di deployment vengano eseguite correttamente.

Il file **deploy1.sh** contiene le istruzioni necessarie per installare tutti i software e le dipendenze richieste per l'avvio del progetto. Questo include l'installazione di pacchetti, configurazioni di sistema, e altre operazioni preliminari indispensabili per il corretto funzionamento dell'applicazione.

Il file **deploy2.sh**, invece, si occupa di avviare Minikube e di effettuare il deployment dell'applicazione su questo cluster Kubernetes. Questo passaggio comprende la creazione dei deployment e dei servizi necessari per eseguire l'applicazione su Minikube.

3 – Conclusioni

Il progetto Shop-Cloud su cloud rappresenta un esempio pratico di come le tecnologie moderne possano essere sfruttate per costruire applicazioni scalabili, resilienti e facilmente gestibili. Questo progetto dimostra come l'integrazione di strumenti e approcci innovativi possa trasformare un'idea in una soluzione tecnologica robusta, garantendo al tempo stesso un'architettura flessibile e pronta a soddisfare esigenze future.

La struttura del progetto si basa sull'adozione di microservizi, sviluppati in Java utilizzando il framework Spring Boot. Questa scelta ha permesso di creare un'architettura distribuita che consente una netta separazione delle responsabilità tra i diversi componenti dell'applicazione. Ogni microservizio è stato progettato per gestire un aspetto specifico del sistema, come la gestione degli utenti, i dati dei prodotti o la gestione delle sessioni. Questa modularità ha migliorato la manutenibilità del codice, facilitando lo sviluppo e la distribuzione di nuove funzionalità.

Per semplificare e standardizzare l'ambiente di esecuzione, i microservizi sono stati containerizzati utilizzando Docker. La containerizzazione ha reso possibile isolare ciascun microservizio in un ambiente indipendente, eliminando conflitti tra le dipendenze e garantendo la consistenza tra gli ambienti di sviluppo, test e produzione. Inoltre, l'uso di Docker ha agevolato il processo di distribuzione, consentendo di creare immagini che sono state poi archiviate e rese accessibili tramite Docker Hub, una piattaforma centralizzata che facilita il versionamento e la condivisione delle immagini Docker.

La gestione e l'orchestrazione dei container sono state realizzate tramite Kubernetes, che ha introdotto un alto grado di automazione e scalabilità all'infrastruttura. Kubernetes ha permesso di distribuire i microservizi su più nodi, bilanciando il carico tra di essi e garantendo l'alta disponibilità del sistema.

Grazie alla sua capacità di scalare automaticamente i container in base al carico di lavoro, Kubernetes ha reso possibile adattare l'applicazione a fluttuazioni improvvise nella domanda, mantenendo prestazioni ottimali anche in situazioni di traffico elevato. Ogni microservizio è stato definito come un Deployment in Kubernetes, con la possibilità di gestire aggiornamenti, rollback e scalabilità in modo indipendente.

Per ospitare l'applicazione nel cloud, il progetto ha utilizzato Amazon Web Services (AWS). Le macchine virtuali EC2 di tipo t3.medium sono state scelte per eseguire i container, grazie al loro ottimo compromesso tra prestazioni e costi. Queste istanze offrono risorse di calcolo sufficienti per supportare i carichi di lavoro dinamici del sistema, con la possibilità di sfruttare il modello basato su crediti della CPU per gestire picchi di traffico senza aumentare i costi operativi. L'infrastruttura cloud di AWS ha fornito una base affidabile e sicura per l'esecuzione dell'applicazione, migliorando ulteriormente la resilienza del sistema.

Un altro aspetto fondamentale del progetto è stata la configurazione della Virtual Private Cloud (VPC) su AWS, che ha garantito un ambiente isolato e sicuro per l'intero sistema. All'interno della VPC, sono stati definiti intervalli di indirizzi IP privati attraverso l'uso di CIDR (Classless Inter-Domain Routing), permettendo una gestione ordinata e scalabile della rete. Sono state configurate subnet per isolare i componenti più sensibili, come i dati degli utenti, e per consentire una comunicazione interna sicura tra i microservizi.

Il sistema è stato progettato per supportare un carico di lavoro dinamico, con la possibilità di scalare in due modalità:

- Scalabilità verticale: aumentando le risorse delle istanze EC2 (ad esempio, più CPU o memoria) per gestire carichi di lavoro più intensi.
- Scalabilità orizzontale: aggiungendo ulteriori istanze EC2 o repliche dei container per distribuire il carico su più risorse.

In conclusione, il progetto Shop-Cloud è un chiaro esempio di come le moderne tecnologie cloud, come Docker, Kubernetes e AWS, possano essere integrate per creare applicazioni avanzate e performanti. L'adozione di un'architettura a microservizi e la containerizzazione hanno garantito un'elevata modularità, semplificando la gestione e il monitoraggio del sistema. La scalabilità, la sicurezza e l'affidabilità ottenute grazie all'integrazione con AWS dimostrano come il cloud computing possa essere un'ottima soluzione per affrontare le sfide dei moderni sistemi software. Questo progetto non solo offre una piattaforma solida per la vendita online, ma rappresenta anche un punto di partenza per ulteriori ottimizzazioni e sviluppi futuri.